Web Developer Portfolio · 2026

춤으로 연결되는
댄스 커뮤니티

춤을 좋아하는 사람들이 정보와 이야기를
자유롭게 나누는 공간

AWS

jiyoungdev.cloud

GitHub

github.com/Jiyoung100/dancetalk

STACK

Java · Spring Boot · MySQL



01

프로젝트 소개

팀원 모집, 강사 구인·구직, 클래스, 대회 및 연습실 정보를 카테고리별로 탐색하고
사용자 간 경험을 공유할 수 있도록 기획한 댄스 커뮤니티입니다.

개발 형태

개인 프로젝트 · 기획부터 배포까지 전체 과정 수행

개발 기간

2026.09 ~ 2026.10

주요 기능

회원, 카테고리, 게시글, 댓글, 추천, 검색, 페이징

배포 환경

AWS Lightsail 인스턴스와
MySQL 데이터베이스를 이용해 웹 서비스 배포

02

개발 환경 및 기술 스택

프로젝트에 사용한 기술과 도구입니다.

BACKEND

- Java
- Spring Boot
- Spring Data JPA
- Spring Security
- MySQL
- Thymeleaf

FRONTEND

- HTML
- CSS
- JavaScript
- Bootstrap

TOOL & DEPLOYMENT

- Spring Tool Suite
- MySQL Workbench
- AWS Lightsail
- Linux Ubuntu

03

주요 기능

Spring Boot를 기반으로 회원가입 · 로그인 후 게시글과 댓글을 작성하고
검색 · 페이징 · 추천으로 게시글을 탐색할 수 있도록 구현했습니다

카테고리 조회

검색 · 페이징

게시글 CRUD

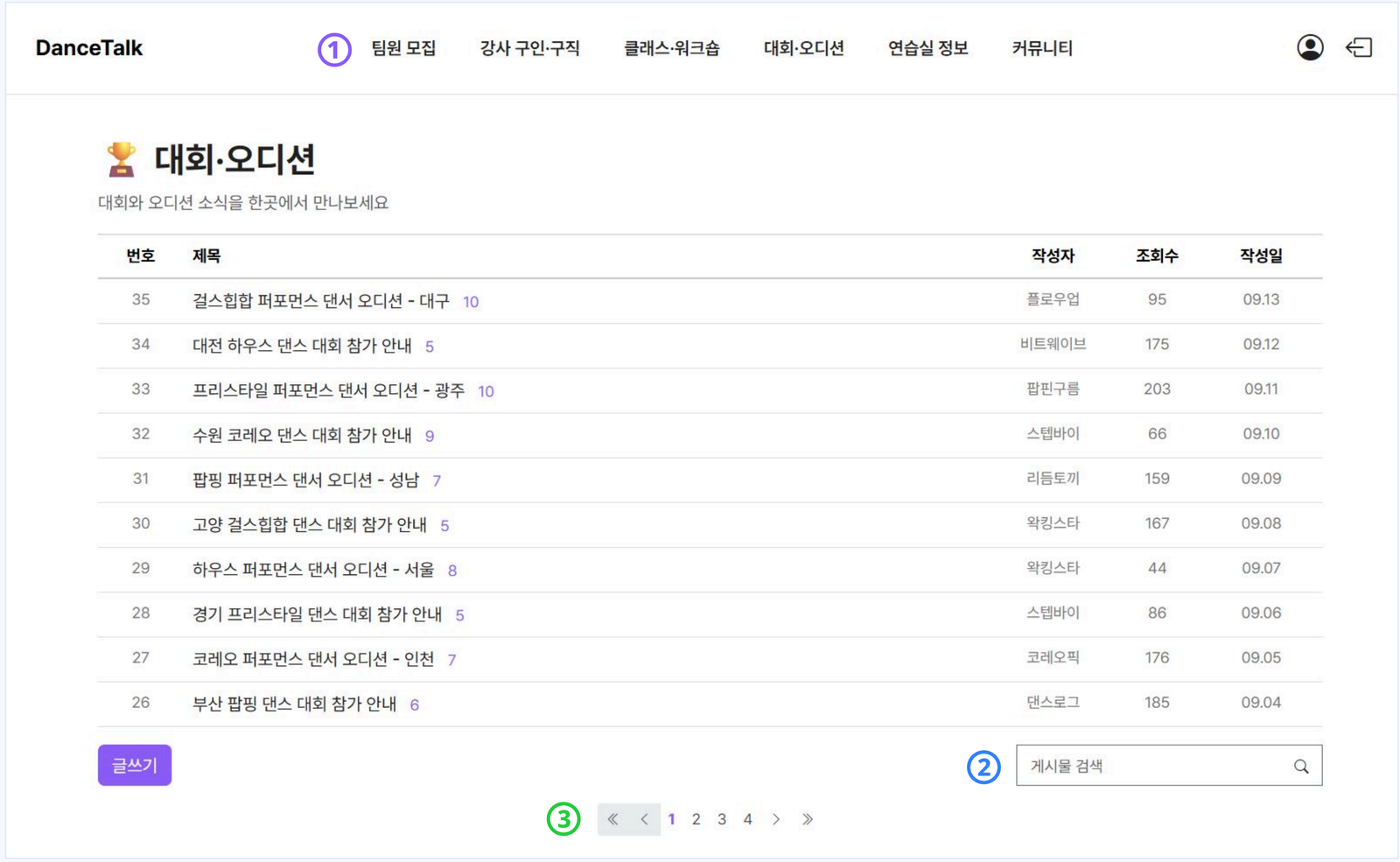
댓글 CRUD

추천 기능

회원 관리



04 카테고리별 게시글 탐색



01 카테고리 조회

Category와 Question을 1:N으로 연결해
선택한 카테고리의 게시글만 조회

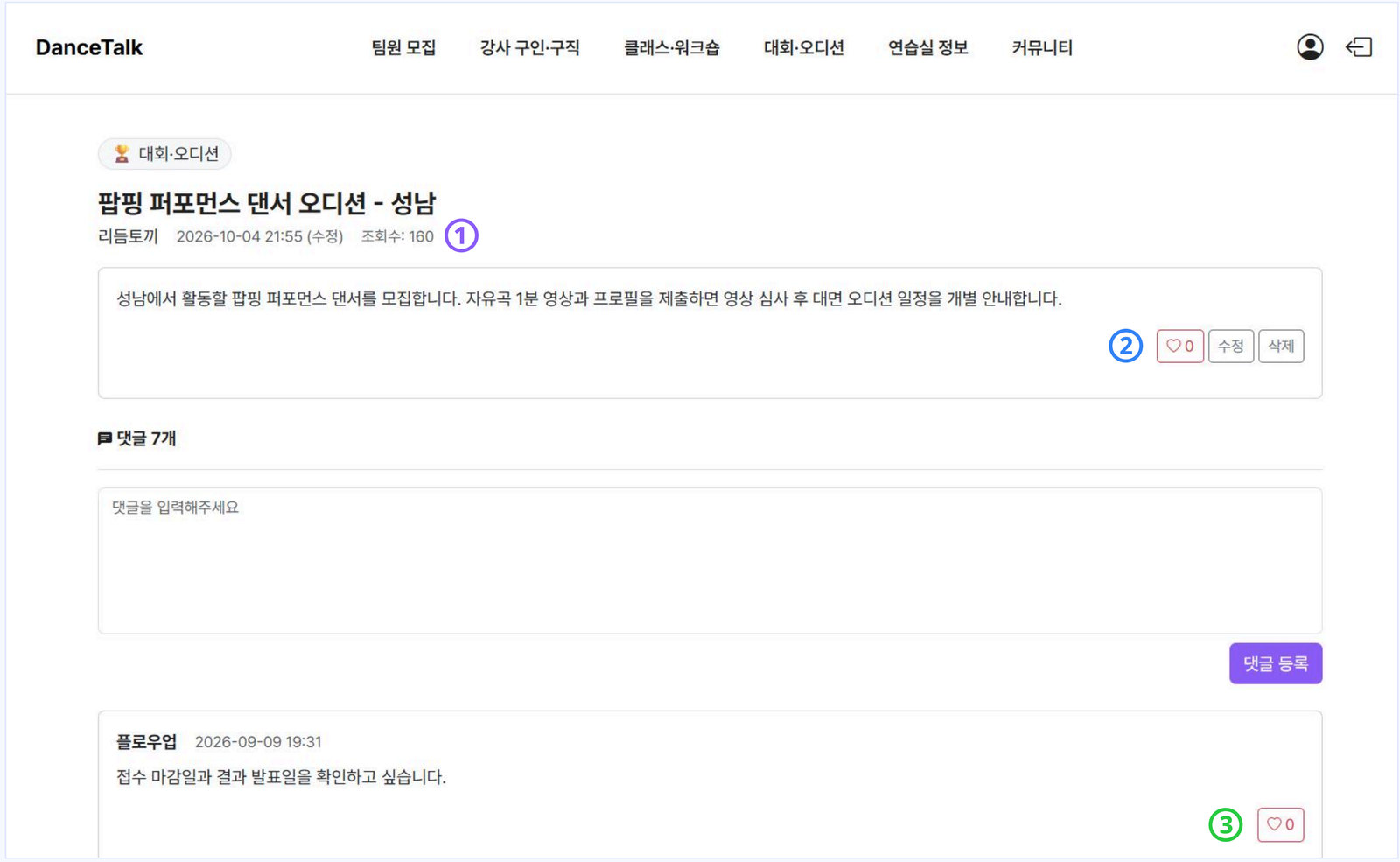
02 검색

제목과 내용을 기준으로
키워드가 포함된 게시글 조회

03 페이징

Pageable을 적용해 한 페이지에 10개의
게시글을 출력하고 최신순 정렬

05 게시글 · 댓글 · 추천



01 조회수

게시글 상세 페이지에 접근할 때
조회수를 1씩 증가

02 게시글 · 댓글 CRUD

로그인 사용자가 게시글과 댓글을 작성하고,
작성자 본인만 수정·삭제할 수 있도록 구현

03 추천 기능

게시글과 댓글에 추천 사용자를 저장하고
Set 자료 구조로 같은 사용자의 중복 추천 방지

06

프로젝트 구조

📁 com.mysite.sbb

› controller	요청을 받아 화면 기능 연결
› service	주요 기능 흐름 처리
› repository	DB 접근 처리
› entity	DB 테이블과 연결되는 클래스
› security	로그인 · 로그아웃 · 권한 설정
› validation	입력값 검증 폼 클래스
› exception	조회 실패 등 예외 처리

선택 이유

요청 처리, 기능 로직, DB 접근 코드가 한 곳에 섞이면
기능이 늘어날수록 수정할 위치를 찾기 어려워집니다.

적용 방식

Controller는 요청과 화면 이동을, Service는 기능 흐름을,
Repository는 DB 접근을 맡도록 역할을 나눴습니다.

이점

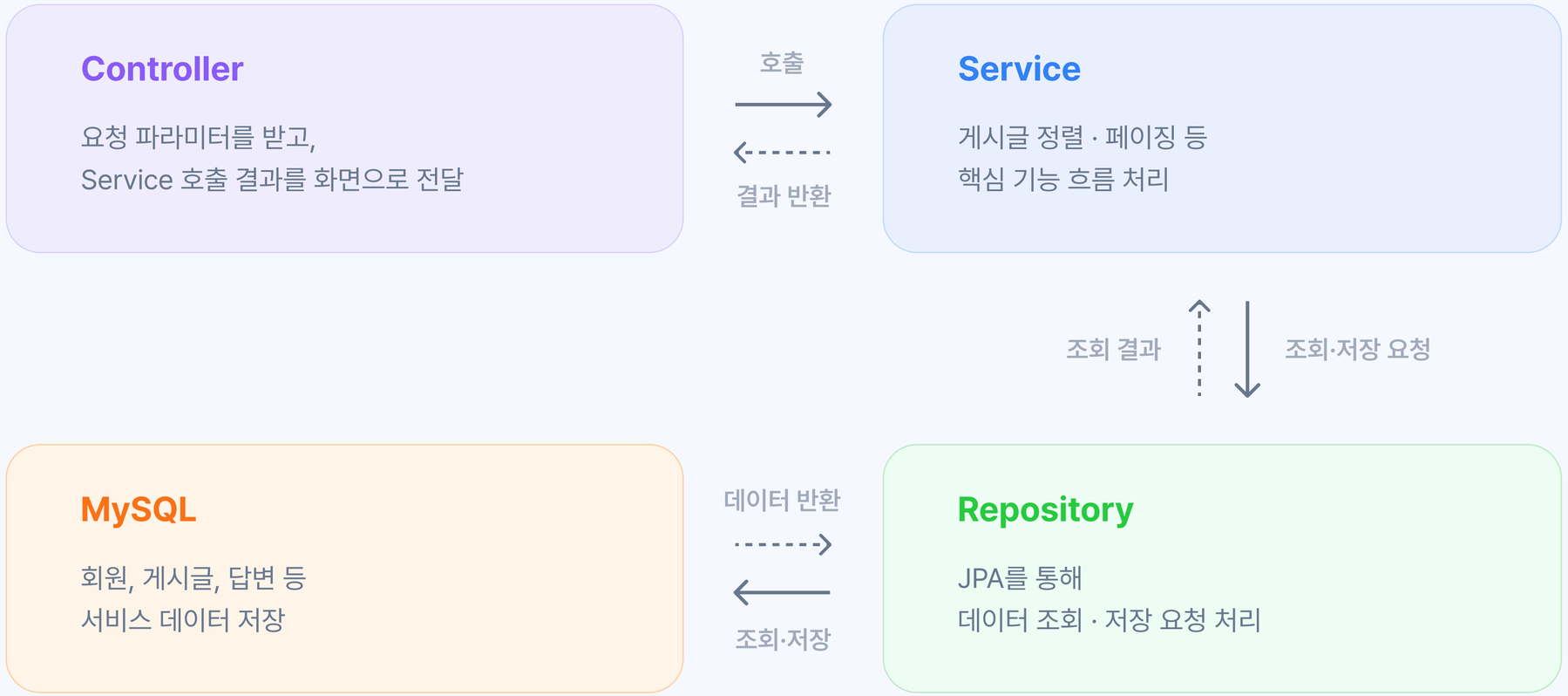
역할이 분리되어 코드 흐름을 파악하기 쉽고,
문제가 생겼을 때 확인할 계층을 빠르게 찾을 수 있었습니다.

Browser → Controller → Service → Repository → DB

07

계층형 구조

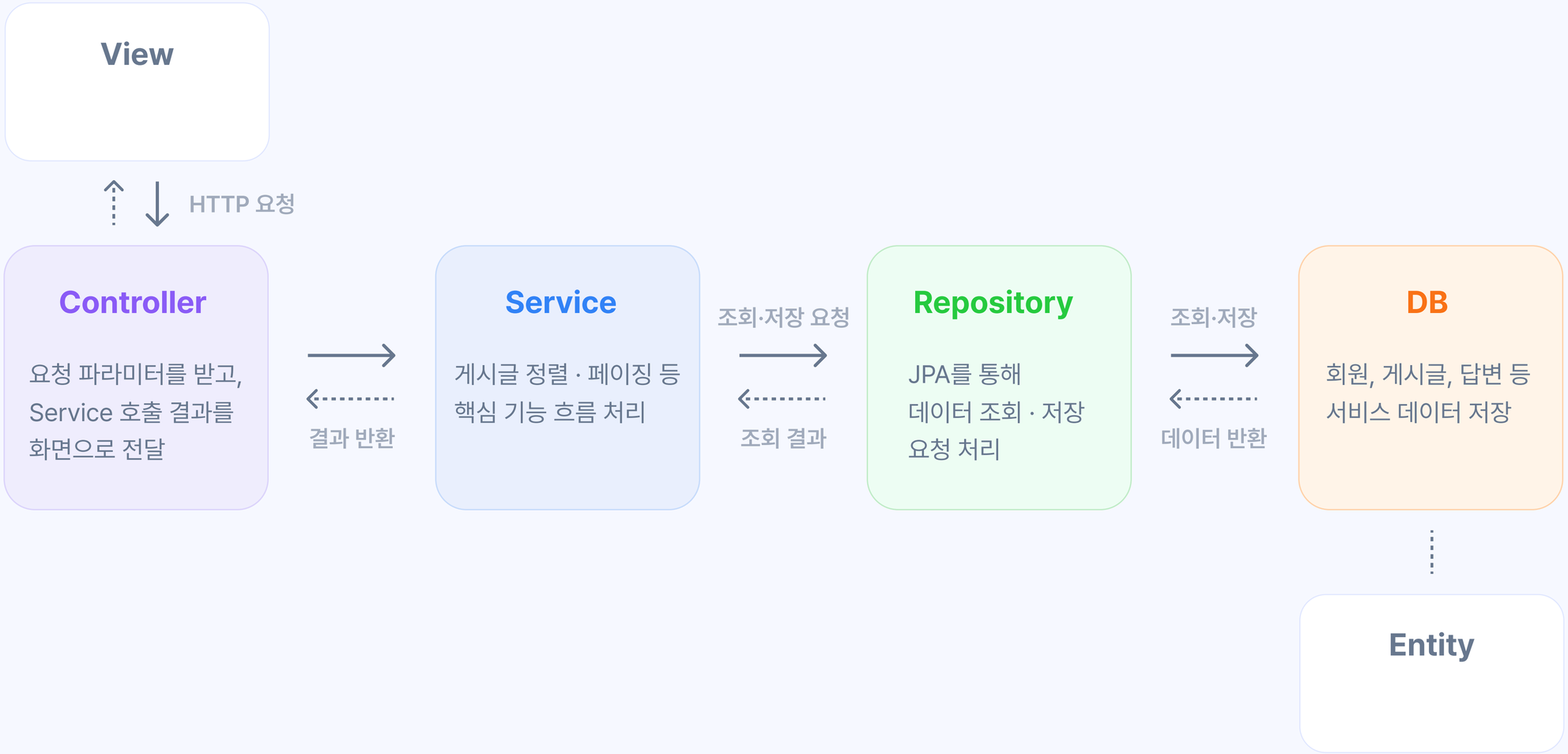
화면 처리, 비즈니스 로직, 데이터 접근 책임을 계층별로 분리해 기능 수정 시 영향 범위를 줄이고
코드 흐름을 쉽게 파악하도록 구성했습니다.



07

계층형 구조

화면 처리, 비즈니스 로직, 데이터 접근 책임을 계층별로 분리해 기능 수정 시 영향 범위를 줄이고
코드 흐름을 쉽게 파악하도록 구성했습니다.



08 계층형 구조 적용 코드

Controller 요청 수신 · 화면 전달

```
@GetMapping("/question/list/{categoryId}")
public String list(@RequestParam(defaultValue = "0") int page,
                   @PathVariable Long categoryId, Model model) {
    Category category = categoryService.getCategory(categoryId);
    Page<Question> paging = questionService.getList(page, categoryId);
    model.addAttribute("category", category);
    model.addAttribute("paging", paging);
    return "question/question_list";
}
```

Service 정렬 · 페이징 처리

```
public Page<Question> getList(int page, Long categoryId) {
    Pageable pageable = PageRequest.of(page, 10, Sort.by(Sort.Direction.DESC, "createDate"));
    return questionRepository.findByCategoryId(categoryId, pageable);
}
```

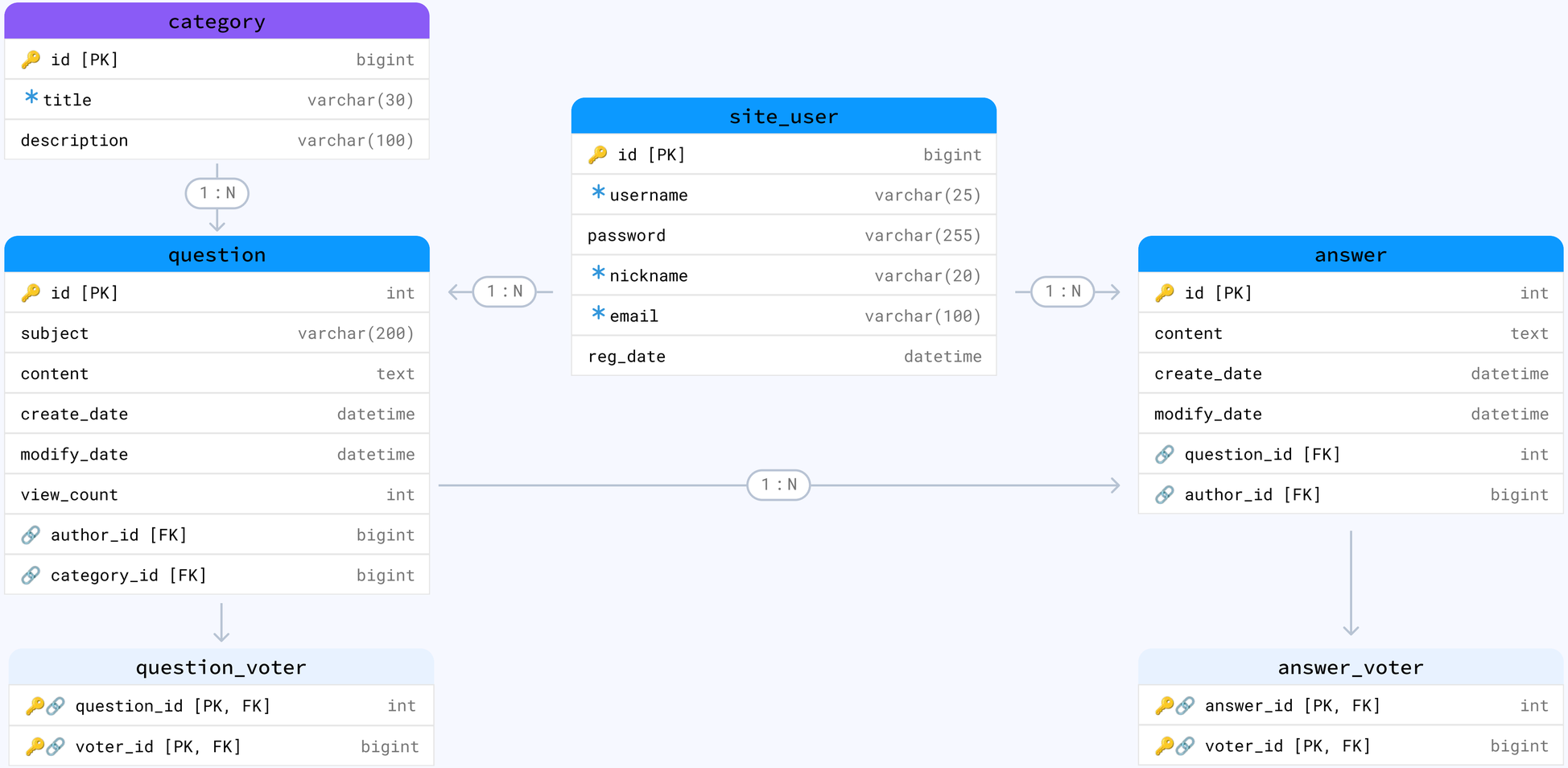
Repository DB 조회

```
public interface QuestionRepository extends JpaRepository<Question, Integer> {
    Page<Question> findByCategoryId(Long categoryId, Pageable pageable);
}
```

09

데이터 설계

Category와 Question을 1:N 관계로 연결해 카테고리별 게시글을 관리했습니다. Answer는 Question을 통해 카테고리를 확인할 수 있으므로 별도의 category_id를 저장하지 않아 데이터 중복을 방지했습니다.



10

핵심 기능 구현과 선택 이유

인증·권한

선택 이유
비회원 접근과 다른 사용자의 게시글 변경 방지

적용 방식
Spring Security 인증 후 로그인 사용자와 작성자 비교

이점
작성자 본인에게만 수정·삭제 권한 부여

```
if (!question.getAuthor().getUsername()
        .equals(principal.getName())) {
    throw new ResponseStatusException(
        HttpStatus.BAD_REQUEST,
        "수정 권한이 없습니다."
    );
}
```

Question–Answer 관계

선택 이유
하나의 게시글에 여러 답변이 작성되는 구조 표현

적용 방식
Question과 Answer를 1:N 관계로 연결

이점
게시글별 답변을 명확하게 조회·관리

```
@OneToMany(mappedBy = "question",
           cascade = CascadeType.REMOVE)
private List<Answer> answerList;


@ManyToOne
private Question question;
```

추천 중복 방지

선택 이유
한 사용자가 같은 게시글을 반복 추천하는 문제 방지

적용 방식
SiteUser를 Set으로 관리하고 중간 테이블에 저장

이점
사용자·게시글 추천 관계를 중복 없이 관리

```
Set<SiteUser> voter = question.getVoter();

if (voter.contains(siteUser)) {
    voter.remove(siteUser);
} else {
    voter.add(siteUser);
}

questionRepository.save(question);
```

11

트러블슈팅

비회원 게시글 상세 접근 오류

문제 비회원의 게시글 상세 페이지 접근 시 화면 렌더링 오류

↓

원인 비회원 Principal에 getUsername() 호출

↓

해결 인증 여부 확인 영역과 작성자 비교 조건 분리

↓

결과 비회원은 조회만 가능하고 수정·삭제는 로그인 사용자에게만 노출

오류 원인

```
#authentication.getPrincipal().getUsername()
```

원인 해결

로그인 사용자에게만 작성자 비교 조건이 실행되도록
인증 여부와 작성자 확인 영역을 분리했습니다

```
<th:block
sec:authorize="isAuthenticated()">
  <a th:if="${question.author != null
        and question.author.username
        == #authentication.name}"
     th:href="@{|/question/modify/
                ${question.id}|}">
    수정
  </a>
</th:block>
```

12

배포 현황

Spring Boot 애플리케이션을 JAR 파일로 빌드한 뒤
AWS Lightsail 서버와 MySQL을 연결하여 웹 서비스로 배포했습니다.



STEP 01 프로젝트 빌드

Gradle을 이용해 실행 가능한 JAR 파일 생성



STEP 02 서버 업로드

빌드된 JAR 파일을 AWS Lightsail 서버에 업로드



STEP 03 애플리케이션 실행

서버에서 JAR 파일을 실행하고 MySQL 연결



STEP 04 도메인 연결

도메인과 HTTPS를 적용해 외부 접속 환경 구성

AWS URL	jiyoungdev.cloud
서비스 상태	● Running
서버	AWS Lightsail
배포 파일	Spring Boot JAR
데이터베이스	MySQL
접속 방식	HTTPS

13

프로젝트를 통해 배운 점

01 구조 설계

Controller–Service–Repository
계층을 분리하며 요청부터
데이터 조회까지 이어지는 흐름을
이해했습니다.

02 데이터 관계

회원·게시글·댓글·추천·카테고리 관계를
설계하며 외래키와 연관관계 관리의
중요성을 배웠습니다.

03 배포와 문제 해결

AWS 배포 과정에서 서버 로그를 확인하고
템플릿 경로, 프로세스 실행,
DB 연결 문제를 해결했습니다.

마무리

기능 구현을 넘어 데이터 관계와 배포 환경까지 직접 설계하고 운영한 프로젝트입니다.



AWS URL https://jiyoungdev.cloud/



GitHub https://github.com/Jiyoung100/dancetalk



Email sjjy0607@naver.com